



RAG Pipelines: Enhancing LLMs with External Knowledge

Bringing the power of retrieval to your AI applications

Why RAG?

Limited Context

LLMs have fixed context windows that restrict the amount of information they can process at once

Knowledge Cutoff

Models trained on static data have a knowledge cutoff date, making them outdated for recent information

Fresh Information

RAG injects **fresh, domain-specific info** when needed

Reduced Hallucinations

External knowledge sources improve accuracy and reduce hallucinations

Core RAG Workflow

1. User Query

Convert user's question into an embedding vector representation

2. Vector DB Search

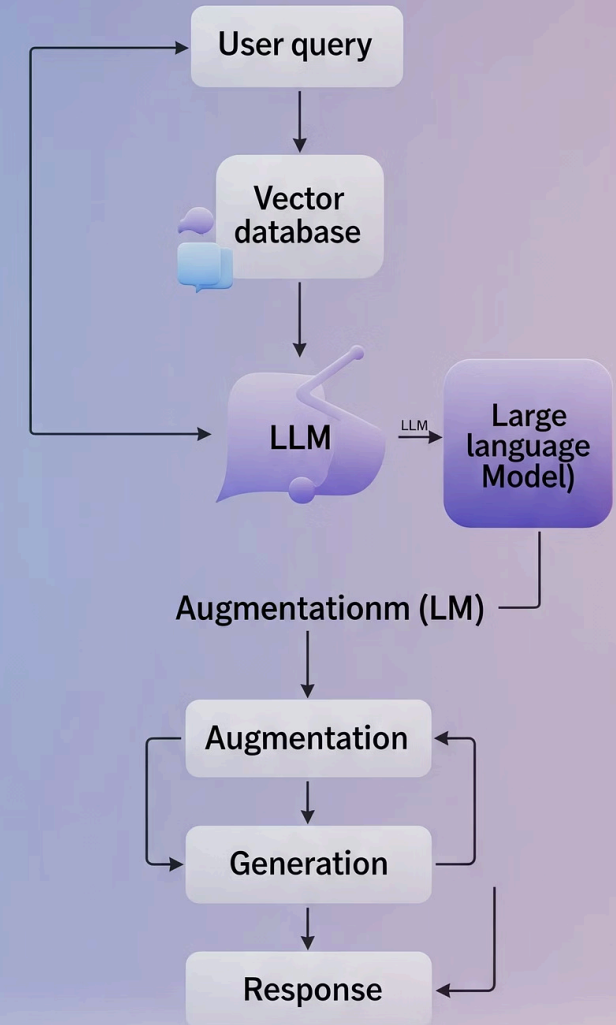
Find semantically relevant document chunks based on vector similarity

3. Context Assembly

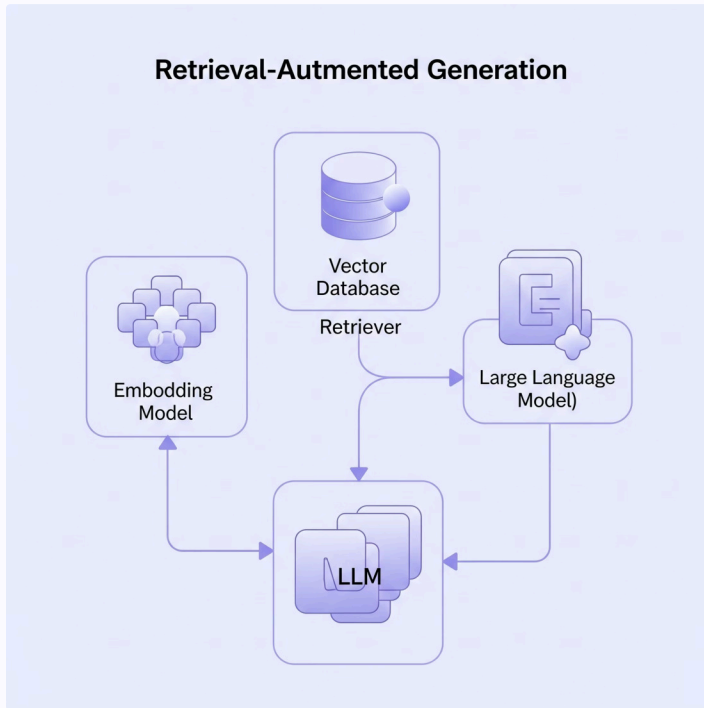
Select and organize top-k results to fit within context window

4. LLM Generation

Combine prompt engineering with retrieved context for accurate responses



Key Components



Embedding Model

Transforms text/images into dense vector representations (e.g., OpenAI, BERT, CLIP)

Vector Database

Stores and indexes embeddings for fast similarity search (FAISS, Pinecone, Weaviate)

Retriever

Manages search, filtering, and reranking of relevant documents

LLM

Generates coherent responses using retrieved context

AI/ML Use Cases



Enterprise Knowledge Assistants

Intelligent systems for HR, IT, and Legal departments that leverage company documentation and policies



Proprietary Document Chatbots

Conversational interfaces that understand and reference your organization's unique content



Scientific Research Assistants

Tools that perform live literature searches and synthesize findings across academic papers



Multimodal RAG

Advanced systems that retrieve and process combinations of text, images, audio, and other data types

Each use case leverages **domain-specific knowledge** to enhance LLM capabilities



Infrastructure Challenges

Scale

Efficiently retrieving from billions of documents requires specialized indexing and sharding strategies

Latency

End-to-end pipeline time includes embedding computation, vector search, and LLM generation

Relevance

Retrieved chunks must be truly relevant, not just superficially similar - noise hurts performance

Freshness

Indexes require continuous or batch updates to incorporate new information

Best Practices

1

Chunking Strategy

Preprocess documents into semantically meaningful chunks of 500-1000 tokens. Preserve context with appropriate overlap between chunks.

2

Hybrid Retrieval

Combine semantic (embedding-based) and keyword (BM25) search for better coverage. Use rerankers to improve precision.

3

Metrics Monitoring

Track relevance scores, recall rates, and hallucination frequencies. Implement human feedback loops to continuously improve.

4

Performance Optimization

Cache frequent queries and their results. Use tiered retrieval for large datasets (coarse then fine search).

Key Takeaways

- RAG bridges the gap between **LLM power** and **fresh knowledge**
- Complete pipeline: Embeddings + Vector DBs + LLMs
- Solves accuracy, relevance, and domain-specific challenges
- Foundation of enterprise-grade generative AI systems

i **Next steps:** Evaluate your use case, choose components, and start with a simple proof of concept

